

一、 判断题

1. 可进行拓扑排序的有向图中，至少存在一个出度为 0 的顶点。
2. Prim 算法将顶点归并，与边数无关，宜采用邻接表作为图的存储表示，适于求稠密网的最小生成树。
3. 已知一颗二叉树的先序序列和后序序列，则能够唯一确定出该二叉树的形状。
4. 从二叉树排序树根结点开始一直沿右侧到最低端叶结点，那么该路径上的结点值一定是从小到大顺序有序的。
5. 具有 n 个终端结点的哈夫曼树共有 $2n-1$ 个结点。
6. 折半查找有序表 (9,11,15,17,24,35,52,76,81,92)，若查找元素 63，将依次与 24、76、52、35 比较，查找结果是失败。
7. 栈和队列都是线性结构，栈只在一端插入和删除，队列则在两端都可以插入和删除。
8. 对于有向图，若一次调用 DFS 算法能够访问图中所有的顶点，则该图是强连通的。
9. 在 n 个结点的单链表中，要在已知结点 *p 之前插入新结点，需要找到它的前驱结点的地址，其时间复杂度为 $O(n)$ 。
10. 树的后序遍历与它转换后的二叉树的后序遍历序列相同。

二、 选择题

1. 若要对 1500 个元素排序，要求既快又稳定，则最好采用 () 方法。
A、直接插入排序
B 堆排序

C 快速排序

D 归并排序

2.利用广义表的 Head () 和 Tail () 的运算, 将元素 a 从广义表 L= (((b, n) f, (a, e)))) 中分离出来, 其运算表达式为 ()。

A Head(Tail(Tail(Tail(Head(Head(L)))))

B Head(Tail(Head(Tail(L)))

C Head(Head(Tail(Head(Head(L)))))

D Head(Head(Tail(Tail(Head(Head(L)))))

3.设有 n 个结点的完全二叉树, 顺序存放在数组 S 【1……n】中, 你们下标为 17 和 19 结点的最近公共祖先的下标是 ()

A 12

B 8

C 4

D 2

4.若一组记录的排序码为 (55,39,76,18,80,24), 则利用希尔排序的方法, 取 d=3 时得到的一趟排序结果为 ()

A 18,39,24,55,80,76

B 24,39,76,18,80,55

C 18,24,39,55,76,80

D 24,39,18,55,80,76

5.若具有 N 个数据元素的查找表的折半查找判定树不是满二叉树，则查找不成功时最少比较 () 次。

A N

B $(\log_2 N) + 1$

C $\log_2 N$

D 1

6.若一个图的边集为 $\{<1,2>, <1,4>, <2,5>, <3,1>, <3,5>, <4,3>\}$ ，则从顶点 1 开始对该图进行深度优先搜索，得到的顶点序列可能为 ()。

A 1,2,5,4,3

B 1,4,3,2,5

C 1,2,5,3,4

D 1,2,3,4,5

7.具有 8 个顶点的非连通无向图最多可有 () 条边。

A 28

B 21

C 8

D 56

8.若用数组 A 【0……6】 来实现循环队列，且当前 front 和 rear 的值分别是 4 和 1，当队列中删除 3 个元素，再加入 2 个元素后，front 和 rear 的值分别为 ()。

- A 3 和 0
- B 0 和 3
- C 6 和 4
- D 4 和 6

9.设四维数组 B 【1..3,2..8,0..5,1..8】 以行主序顺序方法存储在一个连续的存储空间内, 每一个数据元素占 2 个存储单元, 且 B【1,2,0,1】的存储地址是 2000, 则 B 【2,3,4,6】 的存储地址是 ()。

- A 2842
- B 2718
- C 2421
- D 3436

10.给定某二叉树的中序遍历序列为 E B G D A C F H , 后序遍历为 E G D B F H C A, 该二叉树的先序遍历序列为 ()。

- A A B E D G C F H
- B A B E G D C H F
- C A B E D G C H F
- D A B E D G H F C

三、 主观题

1. 某通信系统共有字符：m、n、p、q、r，其取值分别为 8,3,9,5,4。
 - (1) 构造 Huffman 树, (要求所有结点的左孩子的权值不大于右孩子的权值)
 - (2) 设计出各个字符的 Huffman 编码
 - (3) 求出该 Huffman 树的带权路径长度 WPL
 - (4) 译出下列报文：000100111110

2. 已知关键字序列为 (39,29,19,59,69,19,49)，其中，X 表示值为 X 的另一关键字。按如下要求完成本题（计算 ASL 时小数点后保留 3 位）
 - (1) 针对给定关键字序列的不同排序，所构造出的不同形态的二叉排序树中，在最好和最坏的情况下，该二叉排序树的高度各是多少？
 - (2) 根据给定的关键字序列，构造一颗平衡二叉树，分别画出最后 3 个关键字 (69,19,49) 依次插入后各自对应的平衡二叉树的结构。
 - (3) 在等概率的情况下，计算查找成功时该平衡二叉树的平均查找长度 ASL 成功和失败时该平衡二叉树的平均查找长度 ASL 失败。

3. 给定关键字序列 (44,28,53,31,64,49,85,62)，选择哈希函数 $H(\text{Key}) = \text{Key} \bmod 9$ ，处理冲突函数 $H_i = (H(\text{key}) + d_i) \bmod m$ ，其中，m 为哈希表长度， $d_i = 1, 2, 3, 4, 5, \dots$ 。按如下要求完成本题（计算 ASL 时小数点后保留 3 位）。
 - (1) 在 S.Elem【0…11】的存储空间上构造哈希表。

(2) 计算其查找成功时的平均查找长度 ASL 成功

(3) 计算其查找失败时的平均查找长度 ASL 失败

4. 设主串 S='eedededeedeedeedeecdecdecdeeeec',子串 T="eedeeduc".

(1) 求出模式 T 的 Next 【】 的值

(2) 求出模式 T 的 NextVal 【】 的值

(3) 请给出从主串的第二个字符开始的匹配过程，并回答在 S 中查找 T 至少需要几趟匹配？至少需要几次比较？模式匹配成功的位置序号？

5. 若一颗非空 5 叉树上只存在度为 5、3、0 的三种结点，且度为 5 的结点数为 k_5 个，度为 3 的结点数为 k_3 个，叶子结点数为 k_0 个，试证明以下关系成立： $k_0=4*k_5+2*k_3+1$

6. 将给定的关键字序列 (24,42,45,32,63,74,40,67,52) 调整成一个堆，使其满足 $k_i \geq k_{2i}$ 及 $k_i \geq k_{(2i+1)}$ ，一次给出初始堆到输出前 1 个较大关键字的过程中所调整成的每一个堆结构，完成如下填空。

初始堆={—— ———— }

输出第一个元素后的堆={—— ———— , 32,74}

7. 输出结果填空：如下递归程序执行过程中的所有输出。

//SqStack为栈，SqQueue为队列

//InitStack() 构造一个空栈，InitQueue()构造一个空队列

//EnQueue() 入队, DeQueue() 出队
 //Push() 入栈, Pop() 出栈, GetTop() 取栈顶元素
 //StackEmpty() 判断栈是否为空, QueueEmpty() 判断队列是否为空

```
int Proc(SqStack& S, SqQueue& Q) {
    int p, q;
    if (QueueEmpty(Q)) return (0); //判断队列Q是否为空
    p = DeQueue(Q); //出队
    if (p > 1) {
        p = q / 3; //m/3表示m整除3
        EnQueue(Q, q); //入队
        Push(S, q + 1); //入栈
        q = Proc(S, Q) + p; //调用递归函数
    }
    else
        q = Proc(S, Q) - p; //调用递归函数
    printf("%d,%d;", p, q); //打印p, q的值
    return(p);
}

void Sci(SqStack& S, SqQueue& Q, int m, int n) {
    EnQueue(Q, m); //入队
    Push(S, n); //入栈
    n = Proc(S, Q); //调用递归函数
    while (!StackEmpty(S)) { //判断栈是否为空
        m = Pop(S); //出栈
        printf("%d, ", m); //打印m的值
    }
    printf("%d. \n", n); //打印n的值
}

void main() {
    SqStack S; //定义堆栈S
    SqQueue Q; //定义队列S
    InitStack(S); //构造一个空栈S
    InitQueue(Q); //构造一个空队列Q
    Sci(S, Q, 10, 7);
}
```

该递归程序执行过程中的所有输出结果为:

____; ____; ____; ____; ____; ____; ____; ____; ____;

8. 设T为带头结点的单链表的头指针，使用C语言完成如下算法，算法的时间复杂度不超过O (n)

```
typedef struct Node//{
int data;
struct Node* next;
}Node, * Link;

void InsC(Link& T, int c) {
//当链表有奇数个元素结点时，在单链表的正中间位置结点之后，插入数据域值为c/2的结点
//当链表有偶数个元素结点时，在单链表的正中间位置结点之后，插入数据域值为c/3的结点
Link h, g, w;
_____;//前指针h赋初值头指针
_____;//后指针g赋初值头指针
//定位链表正中间位置结点，当前指针h走到表尾时，后指针g正好在正中间位置
while (h) {//循环
    if (_____)//如果不是表尾结点
        _____;//前指针变量移动两步
    else//如果是表尾结点
        _____;//则退出循环
    _____;//后指针变量移动一步
}
_____;//为新结点分配空间，由w指向新结点
if (_____)//奇数个结点
    printf("原链表有奇数个元素结点\n");
    _____;//为新结点数据域赋值
}
else {//偶数个结点
    printf("原链表有偶数个元素结点\n");
    _____;//为新结点数据域赋值
}
_____;//本条语句和下条语句，将新结点插入到正中间结点的后面
_____;
} //InsC
```